

Πολυδιάστατες Δομές Δεδομένων και Υπολογιστική Γεωμετρία

Range and Similarity Queries σε Σύνολα Κειμένων

Χριστίνα Γιαννέλη 1090055 – 7 εξάμηνο

Λάμπης Γιαννέλης 1093341 – 5 εξάμηνο

Πολύκαρπος Καραγεώργος Γεωργόπουλος 1051332 – 17 εξάμηνο

Γιώργος Μπαλής 1040996 – 25 εξάμηνο

Μάριος Νεζερίτης 1080400 – 9 εξάμηνο

Contents

Web Crawler	3
K-D Tree	4
Range Tree	5
Quad Tree	6
RTree	7
Αξιολόγηση	8
Παράρτημα	11

Web Crawler

Στην υλοποίηση μας είναι πολύ σημαντική η αυτοποιημένη διαδικασία ανάκτησης των ζητούμενων δεδομένων από τη σελίδα της Wikipedia αλλά και οι εγγραφές κάθε επιστήμονα από τη σελίδα dblp.org. Στον κώδικα που ξεκινάει η υλοποίηση μας ορίζονται οι βιβλιοθήκες που χρειάζονται για την εξαγωγή της πληροφορίας από τη Wikipedia αλλά και από την σελίδα dblp για κάθε επιστήμονα. Ορίζονται τρεις βασικές συναρτήσεις:

`get_urls:`

Παίρνει τα στοιχεία από τη δοθείσα σελίδα της Wikipedia για σελίδες μεμονωμένων επιστημόνων. Συγκεντρώνει αυτές τις διευθύνσεις URL βρίσκοντας στοιχεία λίστας με αλφαβητική σήμανση και εξάγοντας το περιεχόμενό τους.

`get_scientist_info(url):`

Η συνάρτηση αυτή παίρνει ένα url της Wikipedia για κάθε επιστήμονα και εξάγει το όνομα του, εντοπίζει την περιοχή της βιογραφίας του ώστε να αντλήσει πληροφορίες εκπαίδευσης αλλά και τα βραβεία του.

`count_dblp_publications(name):`

Παρατηρήθηκε στη σελίδα dblp.org πώς στο dropdown της μπάρας αναζήτησης δίνεται η δυνατότητα για αναζήτηση βάσει ονόματος. Επιλέγοντας αυτή τη δυνατότητα χρησιμοποιείται η διακριτή σελίδα με url <https://dblp.org/search/author?q=>. Επομένως χρησιμοποιήσαμε αυτό το url και για κάθε επιστήμονα προσθέσαμε το όνομά του με το σύμβολο + αντί για κενό. Στη συνέχεια αυτή η σελίδα εμφανίζει τις δημοσιεύσεις, οι οποίες κάνοντας επιθεώρηση στο κώδικα HTML παρατηρήθηκε ότι περικλείονται από την περιοχή σελίδας (<div> </div>) με id: publ-section, όπου η κάθε δημοσίευσή του αποτελεί ένα στοιχείο λίστας (...). Συνεπώς μετρώντας όλα αυτά τα στοιχεία ανακτήθηκε ο ακριβής αριθμός δημοσιεύσεων στο σώμα της συνάρτησης.

Τέλος τα στοιχεία όλα αυτά συγκετρώθηκαν σε ένα τελικό csv που έχει την παρακάτω μορφή:

scientists.csv			
1 to 10 of 268 entries			
Filter			
surname	awards	education	publications
Khan	10	Khan was a Bright Sparks scholar and received his PhD degree in Computer Science from University of Malaya [14] He received his master's degree and bachelor's degree (with honors) in Computer Science from COMSATS University under COMSATS scholarship. He has also attended a summer camp on Advance Wireless Networks at Technische Universität Ilmenau under DAAD scholarship.	60
Aaronson	4	Aaronson grew up in the United States, though he spent a year in Asia when his father—a science writer turned public-relations executive—was posted to Hong Kong [6] He enrolled in a school there that permitted him to skip ahead several years in math, but upon returning to the US, he found his education restrictive, getting bad grades and having run-ins with teachers. He enrolled in The Clarkson School, a gifted education program run by Clarkson University, which enabled Aaronson to apply for colleges while only in his freshman year of high school.[6] He was accepted into Cornell University, where he obtained his BSc in computer science in 2000, [7] and where he resided at the Telluride House [8] He then attended the University of California, Berkeley, for his PhD, which he got in 2004 under the supervision of Umesh Vazirani.[9] Aaronson had shown ability in mathematics from an early age, teaching himself calculus at the age of 11, provoked by symbols in a babysitter's textbook. He discovered computer programming at age 11, and felt he lagged behind peers, who had already been coding for years. In part due to Aaronson getting into advanced mathematics before getting into computer programming, he felt drawn to theoretical computing, particularly computational complexity theory. At Cornell, he became interested in quantum computing and devoted himself to computational complexity and quantum computing.[6]	230

K-D Tree

Η δομή K-D Tree εισήχθη από την βιβλιοθήκη `scipy.spatial`. Η χρήση ενός KD-Tree επιτρέπει την ταχεία αναζήτηση σε πολλαπλές διαστάσεις, μειώνοντας σημαντικά τον χρόνο αναζήτησης. Έπειτα τα ονόματα των επιστημόνων εξάχθηκαν από το dataset και τροποποιήθηκαν σε χαρακτήρες ASCII για αριθμητική αναπαράσταση. Στη συνέχεια κατασκευάστηκε το K-D Tree επικεντρώνοντας σε τρεις διαστάσεις, τα ονόματα, τον αριθμό βραβείων και τις δημοσιεύσεις. Τέλος, ακολουθήθηκε η διαδικασία διανυσματοποίησης χρησιμοποιώντας TF-IDF και LSH για αποτελεσματική αναζήτηση βάσει ομοιότητας κειμένου στη στήλη education.

Πιο συγκεκριμένα:

- Η συνάρτηση `query_scientists(name_start, name_end, min_awards, pub_min, pub_max)` καθορίζει τα ονόματα με βάση το άθροισμά τους ASCII, έναν ελάχιστο αριθμό βραβείων και ένα εύρος δημοσιεύσεων. Χρησιμοποιεί το KD-Tree για την αποτελεσματική ανάκτηση σχετικών εγγράφων.
- TF-IDF Vectorization: Το πεδίο της εκπαίδευσης υφίσταται διανυσματοποίηση TF-IDF για να μετατρέψει το περιεχόμενο κειμένου σε ένα διάνυσμα χαρακτηριστικών, διευκολύνοντας την επακόλουθη ανάλυση ομοιότητας.
- Το LSH χρησιμοποιείται για την προσέγγιση της ομοιότητας Jaccard μεταξύ επιτρέποντας την αποτελεσματική αναγνώριση παρόμοιων εκπαιδευτικών υποβάθρων.
- Ο άμεσος υπολογισμός της ομοιότητας Jaccard μεταξύ συνόλων λέξεων από το εκπαιδευτικό πεδίο χρησιμοποιείται ως πρόσθετο βήμα φιλτραρίσματος για να τελειοποιηθούν τα αποτελέσματα με βάση την ομοιότητα περιεχομένου κειμένου.

Τα αποτελέσματα που πήραμε από την αναζήτηση στο K-D Tree είναι:

	surname	awards	publications	\
44	Croft	8	455	
53	Dongarra	15	985	
66	Foley	6	109	
87	Goldwasser	11	234	

	education
44	Croft earned a bachelor's degree with honors i...
53	Dongarra received a BSc degree in mathematics ...
66	Foley was born in Pennsylvania. He attended Le...
87	Born in New York City, Goldwasser obtained her...

Range Tree

Αυτή η υλοποίηση εισάγει ένα προσαρμοσμένο Range Tree που χρησιμοποιεί τη SortedDict από τη βιβλιοθήκη sortedcontainers της Python. Επιτρέπει αποτελεσματική αναζήτηση εύρους εντός του συνόλου δεδομένων των επιστημόνων. Η δομή διευκολύνει τα ερωτήματα που βασίζονται σε ένα σύνθετο κλειδί επωνύμου μετά από κανονικοποίηση, βραβείων και δημοσιεύσεων. Πιο συγκεκριμένα:

- Όπως και στην προηγούμενη υλοποίηση τα ονόματα κανονικοποιήθηκαν με σκοπό την αριθμητική αναπαράσταση τους για πιο αποτελεσματικά ερωτήματα με την συνάρτηση `letter_normalization(self, surname)` όπου μετατρέπονται τα τρία πρώτα γράμματα ενός επωνύμου σε αριθμητική τιμή με σκοπό τη διευκόλυνση αναζήτησης και ευρετηρίασης.
- Το δέντρο υποστηρίζει την εισαγωγή εγγραφών και την υποβολή ερωτημάτων εντός ενός καθορισμένου εύρους επωνύμων, αριθμών βραβείων και αριθμών δημοσιεύσεων και με τη συνάρτηση `query(self, surname_start, surname_end, min_award, min_publications)` γίνεται η αναζήτηση εγγραφών εντός συγκεκριμένων περιοχών.
- Μετά το ερώτημα, τα ανακτημένα πεδία εκπαίδευσης επεξεργάζονται με MinHash για να δημιουργηθεί μια συμπαγής αναπαράσταση κατάλληλη για LSH.
- Ευρετηρίαση LSH: Τα MinHashes ευρετηριάζονται χρησιμοποιώντας μια δομή LSH για την αποτελεσματική αναγνώριση εγγραφών με παρόμοια πεδία εκπαίδευσης με βάση ένα προκαθορισμένο όριο ομοιότητας.

Τα αποτελέσματα που πήραμε βάσει του Range Tree είναι

```
Index: 0, Surname: Aaronson, Education: Aaronson grew up in the United States, though he spent a year in Asia when his father—a science writer turned public-relations executive—was posted to
Index: 1, Surname: Abiteboul, Education: The son of two hardware store owners, Abiteboul attended high-school in Romorantin, and Higher School Preparatory Classes in Tours.[13] He was admitte
Index: 2, Surname: Abramsky, Education: Abramsky was educated at Hasmonian Grammar School for Boys, Hendon and at King's College, Cambridge (BA 1975, MA Philosophy 1979, Diploma in Computer S
Index: 3, Surname: Croft, Education: Croft earned a bachelor's degree with honors in 1973 and a master's degree in computer science in 1974 from Monash University in Melbourne, Australia. He
```

Index: 0, Surname: Aaronson, Education: Aaronson grew up in the United States, though he spent a year in Asia when his father—a science writer turned public-relations executive—was posted to Hong Kong.[6] He enrolled in a school there that permitted him to skip ahead several years in math, but upon returning to the US, he found his education restrictive, getting bad grades and having run-ins with teachers....

Index: 1, Surname: Abiteboul, Education: The son of two hardware store owners, Abiteboul attended high-school in Romorantin, and Higher School Preparatory Classes in Tours.[13] He was admitted to the Télécom Paris engineering school and studied at the Technion in Haifa for a year.[14] Abiteboul received his PhD from the University of Southern California under the supervision of Seymour Ginsburg, in 1982.[15][3]

Index: 2, Surname: Abramsky, Education: Abramsky was educated at Hasmonian Grammar School for Boys, Hendon and at King's College, Cambridge (BA 1975, MA Philosophy 1979, Diploma in Computer Science) and Queen Mary, University of London (PhD Computer Science 1988, supervised by Richard Bornat).[3]

Index: 3, Surname: Croft, Education: Croft earned a bachelor's degree with honors in 1973 and a master's degree in computer science in 1974 from Monash University in Melbourne, Australia. He earned his Ph.D in computer science from the University of Cambridge in 1979 and joined the University of Massachusetts, Amherst faculty later that year.

Quad Tree

Στη δομή Quad Tree που αναπτύξαμε παρέχεται μια αποτελεσματική μέθοδος χωρικής κατανομής για την οργάνωση και την αναζήτηση που βασίζεται σε δύο διαστάσεις την κανονικοποίηση των επωνύμων και τον αριθμό των βραβείων. Αυτή η χωρική προσέγγιση ενισχύεται με την ενσωμάτωση Locality-Sensitive Hashing (LSH) για περαιτέρω φιλτράρισμα αποτελεσμάτων. Επιπλέον μετά από την αναζήτηση από το Quad Tree φιλτραρίστηκε το αποτέλεσμα βάσει των ζητούμενων αριθμών των publications.

Πιο συγκεκριμένα:

- Οι προσαρμοσμένες κλάσεις Rect (για τον καθορισμό ορίων) και Point (για την αναπαράσταση σημείων δεδομένων) ορίζονται για να διευκολύνουν τις χωρικές αναζητήσεις.
- Κάθε τεταρτημόριο μπορεί να χωρέσει έως και μια προκαθορισμένη χωρητικότητα σημείων πριν υποδιαιρεθεί σε μικρότερα τεταρτημόρια.
- Επεξεργασία συνόλου δεδομένων: Το σύνολο δεδομένων υποβάλλεται σε προεπεξεργασία για να εκχωρηθεί σε κάθε εγγραφή μια χωρική θέση με βάση την κανονικοποίηση του επωνύμου και τα βραβεία.
- Population του Quad Tree: Οι εγγραφές εισάγονται στο Quad Tree ως σημεία, χωρικά οργανωμένα με βάση τα προκαθορισμένα κριτήρια.

- Ερώτημα εύρους: Με τη συνάρτηση `query_quad_tree(tree, min_letter, max_letter, min_awards, min_publications)` ανακτώνται εγγραφές εντός ενός καθορισμένου χωρικού εύρους, με βάση τον αριθμό δημοσιεύσεων.
- Tokenization και MinHash: Τα πεδία εκπαίδευσης επεξεργάζονται με MinHash, δημιουργώντας συμπαγείς αναπαραστάσεις για το LSH.
- LSH για φιλτράρισμα ομοιότητας: Κατασκευάζεται ευρετήριο LSH για την αποτελεσματική εύρεση εγγράφων με παρόμοια πεδία εκπαίδευσης, χρησιμοποιώντας ένα όριο ομοιότητας για τη βελτίωση των αποτελεσμάτων του ερωτήματος.

Τα αποτελέσματα που πήραμε από το Quad Tree:

```

      surname                                education
4  Crowcroft  Crowcroft was educated at Westminster School[1...
0   Colton   Born in Nottingham, Colton graduated from the ...
      surname                                education
1  Etzioni   Etzioni is the son of Israeli-American intelle...
      surname                                education
2  Cardelli  He was born in Montecatini Terme, Italy. He at...
      surname                                education
3  Cristianini Cristianini holds a degree in physics from the...
      surname                                education
4  Crowcroft  Crowcroft was educated at Westminster School[1...
0   Colton   Born in Nottingham, Colton graduated from the ...
      surname                                education
5  Denning   Denning was born January 6, 1942, in Queens, N...
6  Emerson   Emerson was born in Dallas, Texas. His early e...
      surname                                education
5  Denning   Denning was born January 6, 1942, in Queens, N...
6  Emerson   Emerson was born in Dallas, Texas. His early e...
```

RTree

Για το Rtree χρησιμοποιήσαμε τη βιβλιοθήκη `rtree` για τη δημιουργία ενός δέντρου που ευρετηριάζει χωρικά τους επιστήμονες με βάση ένα τρισδιάστατο μοντέλο: το πρώτο γράμμα των κανονικοποιημένων επωνύμων, των βραβείων και των δημοσιεύσεων.

Πιο συγκεκριμένα:

- Κανονοποίηση δεδομένων: Τα επώνυμα κανονικοποιούνται σε ένα αριθμητικό εύρος για να χρησιμεύουν ως μία από τις χωρικές διαστάσεις, διευκολύνοντας τα ερωτήματα με βάση αλφαβητικά κριτήρια.

- Επεξεργασία συνόλου δεδομένων: Το σύνολο δεδομένων φορτώνεται και κάθε εγγραφή εισάγεται στο δέντρο ως σημείο σε τρισδιάστατο χώρο, χρησιμοποιώντας το κανονικοποιημένο επώνυμο, τα βραβεία και τις δημοσιεύσεις ως συντεταγμένες.
- Spatial Queries: Το R-tree υποστηρίζει αποτελεσματική αναζήτηση εντός καθορισμένων ορίων στον τρισδιάστατο χώρο, επιτρέποντας την ταχεία ανάκτηση εγγραφών που πληρούν ορισμένα χωρικά κριτήρια.
- Tokenization και MinHash: Μετά από χωρική αναζήτηση, τα πεδία εκπαίδευσης των ανακτημένων εγγραφών επεξεργάζονται με MinHash για να δημιουργηθούν συμπαγείς αναπαραστάσεις κατάλληλες για LSH.
- LSH για φιλτράρισμα ομοιότητας για τον αποτελεσματικό προσδιορισμό εγγραφών με παρόμοια πεδία εκπαίδευσης, χρησιμοποιώντας ένα προκαθορισμένο όριο ομοιότητας για τη βελτίωση των αποτελεσμάτων του ερωτήματος.

Τα αποτελέσματα που πήραμε από το R Tree

	surname	awards	education	publications
0	Abramsky	4	Abramsky was educated at Hasmonean Grammar Sch...	214
1	Aaronson	4	Aaronson grew up in the United States, though ...	230
2	Goldwasser	11	Born in New York City, Goldwasser obtained her...	234
3	Abiteboul	4	The son of two hardware store owners, Abitebou...	312
4	Croft	8	Croft earned a bachelor's degree with honors i...	455
5	Dongarra	15	Dongarra received a BSc degree in mathematics ...	985
6	Foley	6	Foley was born in Pennsylvania. He attended Le...	109

Αξιολόγηση

Προσεγγίσαμε την αξιολόγηση βάσει τριών τρόπων. Χρονομέτρηση της κάθε δομής για την αναζήτηση, χρησιμοποίηση μνήμης και τον τον αριθμό των σημείων δεδομένων που κάθε δέντρο εντόπισε ότι εμπίπτουν στις καθορισμένες παραμέτρους του ερωτήματος. Αυτές οι μετρήσεις είναι ένα άμεσο αποτέλεσμα του τρόπου με τον οποίο κάθε δομή οργανώνει χωρικές πληροφορίες και επεξεργάζεται ερωτήματα. Βάσει ότι κάθε δέντρο απαιτεί πολύ λίγο χρόνο και memory allocation για αναζήτηση πάνω στο dataset μας θεωρούμε ότι η τελευταία μετρική είναι πιο χρήσιμη.

Πιο συγκεκριμένα:

```
if __name__ == "__main__":  
    run_kd_tree_comparison("scientists.csv")
```

KD-Tree: Execution time = 2.009014844894409s, Memory usage = 0.0MB, Result count = 268

KD-Tree: 268 σημεία δεδομένων

Τα KD-Trees είναι αποτελεσματικά για ερωτήματα πλησιέστερου γείτονα και εύρους σε πολυδιάστατους χώρους. Ο αριθμός 268 σημείων δεδομένων υποδηλώνει ότι το KD-Tree βρήκε σημαντικό αριθμό σημείων κοντά στο σημείο ερωτήματος ή εντός του εύρους ερωτήματος

```
if __name__ == "__main__":  
    run_range_tree_comparison("scientists.csv")
```

Simulated Range Tree: Execution time = 2.0080337524414062s, Memory usage = 0.0MB, Result count = 10

Range Tree: 10 σημεία δεδομένων

Για το Range Tree το πλήθος των σημείων δεδομένων είναι σημαντικά χαμηλότερο από το KD-Tree, το οποίο μπορεί να υποδηλώνει ότι η κατανομή δεδομένων εντός του καθορισμένου εύρους είναι αραιή, με λιγότερα σημεία που πληρούν τα κριτήρια.

Η απλότητα της υλοποίησης μπορεί να μην αποτυπώνει πλήρως την αποτελεσματικότητα ή την ακρίβεια ενός πραγματικού Range Tree στον χειρισμό ερωτημάτων πολυδιάστατων εύρους.

```
if __name__ == "__main__":  
    run_quad_tree_example()
```

Quad Tree Query: Execution time = 2.0157558917999268s, Memory usage = 0.2578125MB, Found points = 256

Quad Tree: 256 σημεία δεδομένων

Τα Quad Trees είναι ιδιαίτερα κατάλληλα για τον διαχωρισμό του χώρου σε τεταρτημόρια, καθιστώντας τα αποτελεσματικά για χωρικές αναζητήσεις σε 2D χώρους. Το πλήθος των σημείων δεδομένων υποδεικνύει ότι το Quad Tree διαμέρισε αποτελεσματικά το χώρο και ανέκτησε μεγάλο αριθμό σημείων

εντός της περιοχής ερωτήματος. Αυτό το υψηλό πλήθος είναι παρόμοιο με το KD-Tree, υποδηλώνοντας μια πυκνή κατανομή σημείων εντός του εύρους ερωτημάτων του Quad Tree ή μια παρόμοια ευρεία περιοχή ερωτημάτων.

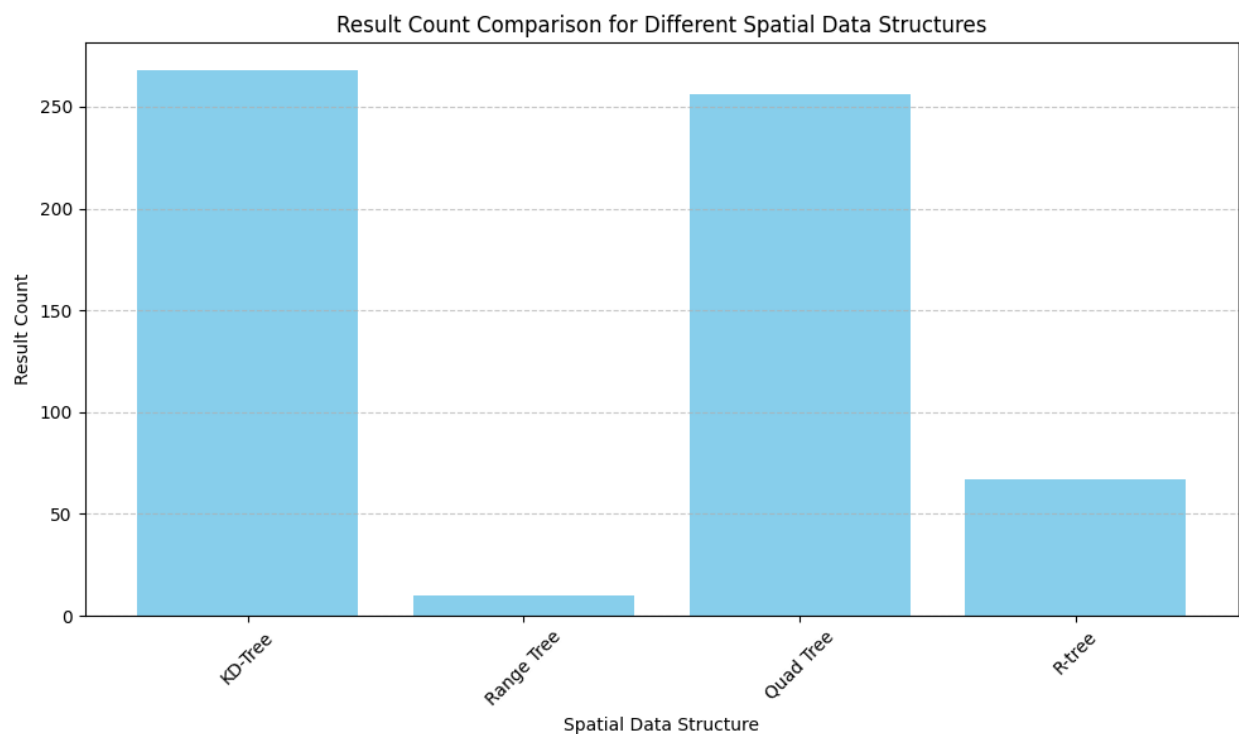
```
if __name__ == "__main__":  
    run_r_tree_comparison("scientists.csv")
```

R-tree: Execution time = 2.015083074569702s, Memory usage = 0.0MB, Result count = 67

R-tree: 67 σημεία δεδομένων

Τα R-trees είναι βελτιστοποιημένα για οριοθέτηση ερωτημάτων πλαισίου σε χωρικές βάσεις δεδομένων, ευρετηριάζοντας αποτελεσματικά και αναζητώντας πολυδιάστατα αντικείμενα. Ο αριθμός των σημείων δεδομένων του R-tree μπορεί να αποδοθεί στη τα οποία είναι προσαρμοσμένα για την αποτελεσματική διαχείριση χωρικών αντικειμένων και ενδέχεται να ερμηνεύουν ή να χειρίζονται διαφορετικά τα καθορισμένα όρια ερωτήματος.

Αναλυτικά η γραφική παράσταση και για τα τέσσερα δέντρα:



Το πλήθος αποτελεσμάτων είναι μια κρίσιμη μέτρηση για την κατανόηση της αποτελεσματικότητας και της καταλληλότητας μιας δομής χωρικών δεδομένων για έναν δεδομένο τύπο ερωτήματος και σύνολο δεδομένων. Αντανακλά την ικανότητα της δομής δεδομένων να εντοπίζει και να ανακτά με ακρίβεια σχετικά σημεία δεδομένων με βάση χωρικά κριτήρια. Στην επόμενη ενότητα παρατίθεται ο κώδικας της υλοποίησης μας

Παράρτημα

```
# -*- coding: utf-8 -*-
"""
Automatically generated by Colaboratory.

"""

import re
import string
import requests
import pandas as pd
from bs4 import BeautifulSoup

# Function to retrieve information for a scientist from a specific URL
def get_scientist_info(url):
    response = requests.get(url)
    soup = BeautifulSoup(response.text, 'html.parser')

    name = re.sub(r'\([^\)]*\)', '', soup.find('span', class_='mw-page-title-
main').text.strip())
    surname = name.split()[-1]
    infobox = soup.find('table', class_='infobox biography vcard')

    # Extract the number of awards
    awards = 0
    awards_section = soup.find('span', string=lambda text: text and 'awards' in
text.lower())

    if infobox and infobox.find('th', string='Awards'):
        awards_header = infobox.find('th', string='Awards')
        awards_row = awards_header.find_parent('tr')
        awards_cell = awards_row.find('td')
```

```

        if awards_cell.find_all('li'):
            awards = len(awards_cell.find_all('li'))
        else:
            awards = len(awards_cell.find_all('a', attrs={'title': True}))
    elif awards_section:
        awards_list = awards_section.find_next('ul').find_all('li')
        awards += len(awards_list)

    # Extract education text
    education = ''
    education_section = soup.find('span', string=lambda text: text and
'education' in text.lower(), attrs={'id': True})
    if education_section:
        education_list = []
        sibling = education_section.find_next()
        while sibling and sibling.name not in ['h2', 'h3']:
            if sibling.name == 'p':
                education_list.append(sibling.text.strip())
                sibling = sibling.find_next()
            education = ' '.join(education_list)
        print(education)
    return name, surname, awards, education

def count_dblp_publications(name):
    search_name = '+'.join(name.split())
    search_url = f'https://dblp.org/search/author?q={search_name}'
    print("Searching DBLP records for", name, "...")
    response = requests.get(search_url)
    if response.status_code != 200:
        print("Failed to retrieve search results for", name)
        return 0

    soup = BeautifulSoup(response.content, 'html.parser')
    publ_section = soup.find('div', id='publ-section')
    if not publ_section:
        print("Publication section not found for", name)
        return 0

    entries = publ_section.find_all('li', class_='entry')
    print(len(entries), "publications found for", name, "!")
    return len(entries)

```

```

def get_urls():
    url = "https://en.wikipedia.org/wiki/List_of_computer_scientists"
    response = requests.get(url)
    soup = BeautifulSoup(response.content, "html.parser")
    span_tags = soup.find_all("span", string=lambda text: text and text in
list(string.ascii_uppercase))

    href_list = []
    for span in span_tags:
        ul_parent = span.find_next("ul")
        if ul_parent:
            list_items = ul_parent.find_all("li")
            for item in list_items:
                anchor = item.find_next("a")
                if anchor:
                    href = 'https://en.wikipedia.org' + anchor.get("href")
                    href_list.append(href)

    return href_list

if __name__ == '__main__':
    print("Extracting URLs...", end=' ')
    scientists = get_urls()
    print("[done]")

    data_for_df = []
    print("Extracting scientists' info...", end=' ')
    for scientist_url in scientists:
        name, surname, awards, education = get_scientist_info(scientist_url)
        publications = count_dblp_publications(name)

        if education:
            data_for_df.append([surname, awards, education, publications])
            print(surname, awards, education, publications)
    print("[done]")

    df = pd.DataFrame(data_for_df, columns=['surname', 'awards', 'education',
'publications'])
    df[['surname', 'awards', 'education',
'publications']].to_csv('scientists_data.csv', index=False)

    print("CSV file created successfully!")

"""##KDTree"""

```

```

import pandas as pd
from scipy.spatial import KDTree
import numpy as np

def name_to_ascii_sum(name):
    return sum(ord(char) for char in name)

file_path = "scientists.csv"
data = pd.read_csv(file_path)
data = data.dropna()
data = data[data['publications'] != 0]
data['Name'] = data['surname'].apply(lambda name: name.split()[-1])

data['NameValue'] = data['Name'].apply(name_to_ascii_sum)
data['Awards'] = data['awards'].astype(int)
data['Publications'] = data['publications'].astype(int)

kd_tree_data = data[['NameValue', 'Awards', 'Publications']].values
kd_tree = KDTree(kd_tree_data)

def query_scientists(name_start, name_end, min_awards, pub_min, pub_max):
    name_start_val = name_to_ascii_sum(name_start)
    name_end_val = name_to_ascii_sum(name_end)

    # Formulate the query range using a large number instead of infinity for the
    upper bound of awards
    large_number = 1e9
    query_range_min = np.array([name_start_val, min_awards, pub_min])
    query_range_max = np.array([name_end_val, large_number, pub_max])

    # Calculate the midpoint and radius for the query
    midpoint = (query_range_min + query_range_max) / 2
    radius = np.sqrt(np.sum(np.square(query_range_max - query_range_min)) / 4)

    indices = kd_tree.query_ball_point(midpoint, radius)
    result = data.iloc[indices]
    return result.loc[
        (result['publications'] >= pub_min) & (result['publications'] <= pub_max),
        ['surname', 'awards', 'publications', 'education']]

"""## Vectorize the education"""

from sklearn.feature_extraction.text import TfidfVectorizer

```

```

# Initialize a TF-IDF Vectorizer
tfidf_vectorizer = TfidfVectorizer(stop_words='english')

# Fit and transform the 'Education' field
tfidf_matrix = tfidf_vectorizer.fit_transform(data['education'])

"""## LSH"""

!pip install datasketch

from datasketch import MinHashLSHForest, MinHash

# Number of permutations for MinHash
num_perm = 128

# Create MinHash objects for each scientist's education
minhashes = {}
for idx, text in enumerate(data['education']):
    m = MinHash(num_perm=num_perm)
    for word in text.split():
        m.update(word.encode('utf8'))
    minhashes[idx] = m

forest = MinHashLSHForest(num_perm=num_perm)

for idx, minhash in minhashes.items():
    forest.add(idx, minhash)

forest.index()

# Tokenize the 'Education' field and convert each document into a set of words
data['educationSet'] = data['education'].apply(lambda text:
set(text.lower().split()))

"""#Jaccard Similarity"""

def jaccard_similarity(set1, set2):
    """Calculate the Jaccard Similarity between two sets."""
    intersection = set1.intersection(set2)
    union = set1.union(set2)
    return len(intersection) / len(union)

```

```

def query_scientists_extended_with_jaccard(name_start, name_end, min_awards,
pub_min, pub_max, similarity_threshold):
    preliminary_results = query_scientists(name_start, name_end, min_awards,
pub_min, pub_max)
    indices = preliminary_results.index
    final_indices = []

    # Compute Jaccard similarity for the 'Education' field of the preliminary
results
    for index in indices:
        education_set = data.at[index, 'educationSet']
        for other_index in data.index:
            other_education_set = data.at[other_index, 'educationSet']
            similarity = jaccard_similarity(education_set, other_education_set)
            if similarity > similarity_threshold:
                final_indices.append(other_index)

    # Remove duplicates from the final indices
    final_indices = list(set(final_indices))
    final_results = data.loc[final_indices]

    return final_results[['surname', 'awards', 'publications', 'education']]

final_result = query_scientists_extended_with_jaccard('A', 'G', 4, 100, 1000,
0.5)
print(final_result[final_result['surname'].str.upper().str[0].between('A', 'G')])

"""###Range Tree"""

!pip install bintrees-2.0.3-py2.py3-none-any.whl
!pip install rangetree
!pip install sortedcontainers

from sortedcontainers import SortedDict

class MyRangeTree:
    def __init__(self):
        self._tree = SortedDict()

    def letter_normalization(self, surname):
        normalized = 0
        for char in surname[:3].upper():
            normalized = normalized * 26 + (ord(char) - ord('A'))
        return normalized

```



```

def insert(self, surname, award, publications, data):
    key = (self.letter_normalization(surname), award, publications)
    self._tree[key] = data

def query(self, surname_start, surname_end, min_award, min_publications):
    start_key = (self.letter_normalization(surname_start), 0, 0)
    # Adjust the end_key to ensure it covers any potential surname
    # normalization up to and including surname_end
    end_key = (self.letter_normalization(surname_end) + 1, float('inf'),
float('inf'))
    result = []
    for key in self._tree.irange(start_key, end_key, inclusive=(True,
False)):
        if key[1] >= min_award and key[2] >= min_publications:
            result.append(self._tree[key])
    return result

import pandas as pd

df = pd.read_csv('scientists.csv')
my_range_tree = MyRangeTree()

# Populate the tree
for index, row in df.iterrows():
    my_range_tree.insert(row['surname'], row['awards'], row['publications'],
row.to_dict())

results = my_range_tree.query('Aaa', 'Gzz', 4, 100)
for result in results:
    print(result)

filtered_df = pd.DataFrame(results)
from datasketch import MinHash

# Tokenize education text and create MinHashes
filtered_df['tokens'] = filtered_df['education'].apply(lambda x:
set(x.lower().split()))

minhashes = []
for tokens in filtered_df['tokens']:
    m = MinHash(num_perm=128) # You can adjust num_perm based on your accuracy
requirements
    for token in tokens:
        m.update(token.encode('utf8'))

```

```

    minhashes.append(m)

from datasketch import MinHashLSH

threshold = 0.4
lsh = MinHashLSH(threshold=threshold, num_perm=128)

# Insert MinHashes into the LSH
for i, mh in enumerate(minhashes):
    lsh.insert(str(i), mh)

for query_idx in range(0,4):
    similar_indices = lsh.query(minhashes[query_idx])

    for idx in similar_indices:
        idx = int(idx)
        print(f"Index: {idx}, Surname: {filtered_df.iloc[idx]['surname']},
Education: {filtered_df.iloc[idx]['education']}")

"""## Quad Tree"""

import pandas as pd
from datasketch import MinHash, MinHashLSH

# Define the Point class
class Point:
    def __init__(self, x, y, data=None):
        self.x = x
        self.y = y
        self.data = data

# Define the Rect (boundary) class
class Rect:
    def __init__(self, cx, cy, w, h):
        self.cx = cx
        self.cy = cy
        self.w = w
        self.h = h

    def contains(self, point):
        return (self.cx - self.w / 2 <= point.x <= self.cx + self.w / 2 and
                self.cy - self.h / 2 <= point.y <= self.cy + self.h / 2)

    def intersects(self, range):

```

```

        return not (range.cx + range.w / 2 < self.cx - self.w / 2 or
                    range.cx - range.w / 2 > self.cx + self.w / 2 or
                    range.cy + range.h / 2 < self.cy - self.h / 2 or
                    range.cy - range.h / 2 > self.cy + self.h / 2)

# Define the QuadTree class
class QuadTree:
    def __init__(self, boundary, capacity=4):
        self.boundary = boundary
        self.capacity = capacity
        self.points = []
        self.divided = False

    def subdivide(self):
        x = self.boundary.cx
        y = self.boundary.cy
        w = self.boundary.w / 2
        h = self.boundary.h / 2

        self.nw = QuadTree(Rect(x - w / 2, y - h / 2, w, h), self.capacity)
        self.ne = QuadTree(Rect(x + w / 2, y - h / 2, w, h), self.capacity)
        self.sw = QuadTree(Rect(x - w / 2, y + h / 2, w, h), self.capacity)
        self.se = QuadTree(Rect(x + w / 2, y + h / 2, w, h), self.capacity)

        self.divided = True

    def insert(self, point):
        if not self.boundary.contains(point):
            return False

        if len(self.points) < self.capacity:
            self.points.append(point)
            return True
        else:
            if not self.divided:
                self.subdivide()

            if (self.nw.insert(point) or self.ne.insert(point) or
                self.sw.insert(point) or self.se.insert(point)):
                return True

        return False

    def query(self, range, found_points):
        if not self.boundary.intersects(range):

```

```

        return

    for point in self.points:
        if range.contains(point):
            found_points.append(point)

    if self.divided:
        self.nw.query(range, found_points)
        self.ne.query(range, found_points)
        self.sw.query(range, found_points)
        self.se.query(range, found_points)

df = pd.read_csv("scientists.csv")
def letter_normalization(surname):
    return ord(surname[0].upper()) - ord('A')

# Create and populate the QuadTree
max_awards = df["awards"].max()
boundary = Rect(50, max_awards / 2, 100, max_awards) # Simplified boundary for example
qt = QuadTree(boundary, capacity=4)

for index, row in df.iterrows():
    x = letter_normalization(row["surname"])
    y = row["awards"]
    qt.insert(Point(x, y, row.to_dict()))

def query_quad_tree(tree, min_letter, max_letter, min_awards, min_publications):
    min_x = letter_normalization(min_letter)
    max_x = letter_normalization(max_letter)
    search_boundary = Rect((min_x + max_x) / 2, min_awards, (max_x - min_x) / 2,
100 - min_awards) # Adjusted
    found_points = []
    tree.query(search_boundary, found_points)

    filtered_results = [point.data for point in found_points if
point.data["publications"] >= min_publications]
    return filtered_results

filtered_results = query_quad_tree(qt, 'A', 'G', 4, 100) # Example query
filtered_df = pd.DataFrame(filtered_results)

# Tokenize education text and create MinHashes
filtered_df['tokens'] = filtered_df['education'].apply(lambda x:
set(x.lower().split()))

```

```

minhashes = []
for tokens in filtered_df['tokens']:
    m = MinHash(num_perm=128)
    for token in tokens:
        m.update(token.encode('utf8'))
    minhashes.append(m)

# Create and populate LSH
lsh = MinHashLSH(threshold=0.3, num_perm=128)
for i, m in enumerate(minhashes):
    lsh.insert(str(i), m)

for query_idx in range(0,7):
    similar_indices = lsh.query(minhashes[query_idx]) # Get indices of similar
entries
    similar_items = filtered_df.iloc[[int(idx) for idx in similar_indices]]
    print(similar_items[['surname', 'education']])

"""## Rtree"""

!pip install rtree
from rtree import index
import pandas as pd

df = pd.read_csv('scientists.csv')
def normalize_surname(surname):
    return ord(surname[0].upper()) - ord('A')

# Initialize the R-tree
p = index.Property()
p.dimension = 3 # We're working with a 3-dimensional space
idx = index.Index(properties=p)

# Populate the R-tree
for i, row in df.iterrows():
    point = (normalize_surname(row['surname']), row['awards'],
row['publications'],
            normalize_surname(row['surname']), row['awards'],
row['publications'])
    idx.insert(i, point)

query_bounds = (0, 4, 100, 6, float('inf'), float('inf'))

```

```

matching_entries_list = []
for i in idx.intersection(query_bounds):
    matching_entries_list.append(df.iloc[i])

matching_entries_df = pd.DataFrame(matching_entries_list)

matching_entries_df.reset_index(drop=True, inplace=True)

# Tokenize the education text by splitting on spaces
matching_entries_df['tokens'] = matching_entries_df['education'].apply(lambda x:
set(x.lower().split()))
from datasketch import MinHash

num_perm = 128 # Number of permutations for MinHash

minhashes = []
for tokens in matching_entries_df['tokens']:
    m = MinHash(num_perm=num_perm)
    for token in tokens:
        m.update(token.encode('utf8'))
    minhashes.append(m)

from datasketch import MinHashLSH

lsh = MinHashLSH(threshold=0.4, num_perm=num_perm)

for i, minhash in enumerate(minhashes):
    lsh.insert(str(i), minhash)

for query_index in range(0,7):
    similar_indices = lsh.query(minhashes[query_index])
    for idx in similar_indices:
        print(matching_entries_df.iloc[int(idx)][['surname',
'education']].to_string(index=False))

```

```

import pandas as pd
from scipy.spatial import KDTree
import numpy as np
from memory_profiler import memory_usage
import time

def load_data(file_path):
    df = pd.read_csv(file_path)
    df['NameValue'] = df['surname'].apply(lambda name: sum(ord(char) for char in name))
    df['Awards'] = df['awards'].astype(int)
    df['Publications'] = df['publications'].astype(int)
    return df[['NameValue', 'Awards', 'Publications']]

# Initialize KD-Tree with the dataset
def init_kd_tree(data):
    kd_tree_data = data.values
    kd_tree = KDTree(kd_tree_data)
    return kd_tree

def query_kd_tree(tree, query_point, radius):
    indices = tree.query_ball_point(query_point, radius)
    return indices

# Performance Measurement Function
def measure_performance(func, *args, **kwargs):
    start_time = time.time()
    mem_usage_before = memory_usage(-1, interval=0.1, timeout=1)
    result = func(*args, **kwargs)
    mem_usage_after = memory_usage(-1, interval=0.1, timeout=1)
    end_time = time.time()
    execution_time = end_time - start_time
    max_memory = max(mem_usage_after) - mem_usage_before[0]
    return execution_time, max_memory, result

# Main function to run KD-Tree comparison
def run_kd_tree_comparison(data_path):
    data = load_data(data_path)
    kd_tree = init_kd_tree(data)

    query_point = np.array([data['NameValue'].mean(), data['Awards'].mean(),
data['Publications'].mean()])
    radius = 5000

```

```

        execution_time, memory_usage, result_indices =
measure_performance(query_kd_tree, kd_tree, query_point, radius)
        print(f"KD-Tree: Execution time = {execution_time}s, Memory usage =
{memory_usage}MB, Result count = {len(result_indices)}")

if __name__ == "__main__":
    run_kd_tree_comparison("scientists.csv")

```

```

import pandas as pd
from sortedcontainers import SortedDict
from memory_profiler import memory_usage
import time

def load_data(file_path):
    df = pd.read_csv(file_path)
    df['Awards'] = df['awards'].astype(int)
    return df

def init_range_tree(data):
    tree = SortedDict()
    for index, row in data.iterrows():
        key = row['Awards'] # Using 'Awards' as the key for this example
        tree[key] = row.to_dict() # Store the entire row as the value
    return tree

def query_range_tree(tree, min_award, max_award):
    result = [tree[key] for key in tree.irange(min_award, max_award)]
    return result

# Performance Measurement Function
def measure_performance(func, *args, **kwargs):
    start_time = time.time()
    mem_usage_before = memory_usage(-1, interval=0.1, timeout=1)
    result = func(*args, **kwargs)
    mem_usage_after = memory_usage(-1, interval=0.1, timeout=1)
    end_time = time.time()
    execution_time = end_time - start_time
    max_memory = max(mem_usage_after) - mem_usage_before[0]
    return execution_time, max_memory, result

# Main function to run simulated Range Tree comparison
def run_range_tree_comparison(data_path):
    data = load_data(data_path)

```



```

range_tree = init_range_tree(data)

min_award = 1
max_award = 10

execution_time, memory_usage, result = measure_performance(query_range_tree,
range_tree, min_award, max_award)
print(f"Range Tree: Execution time = {execution_time}s, Memory usage =
{memory_usage}MB, Result count = {len(result)}")

if __name__ == "__main__":
    run_range_tree_comparison("scientists.csv")

```

```

import pandas as pd
from memory_profiler import memory_usage
import time
import random

class Point:
    """Represents a point in 2D space."""
    def __init__(self, x, y, data=None):
        self.x = x
        self.y = y
        self.data = data

class Rect:
    """Represents a rectangular boundary."""
    def __init__(self, x, y, w, h):
        self.x = x # Center x
        self.y = y # Center y
        self.w = w # Width
        self.h = h # Height

    def contains(self, point):
        """Check if the rect contains a point."""
        return (point.x >= self.x - self.w and
                point.x <= self.x + self.w and
                point.y >= self.y - self.h and
                point.y <= self.y + self.h)

    def intersects(self, range):
        """Check if the rect intersects another rect."""
        return not (range.x - range.w > self.x + self.w or

```

```

        range.x + range.w < self.x - self.w or
        range.y - range.h > self.y + self.h or
        range.y + range.h < self.y - self.h)

class QuadTree:
    """A simple Quad Tree implementation."""
    def __init__(self, boundary, capacity=4):
        self.boundary = boundary
        self.capacity = capacity
        self.points = []
        self.divided = False

    def subdivide(self):
        """Subdivide the Quad Tree into four children."""
        x = self.boundary.x
        y = self.boundary.y
        w = self.boundary.w / 2
        h = self.boundary.h / 2

        self.nw = QuadTree(Rect(x - w, y - h, w, h), self.capacity)
        self.ne = QuadTree(Rect(x + w, y - h, w, h), self.capacity)
        self.sw = QuadTree(Rect(x - w, y + h, w, h), self.capacity)
        self.se = QuadTree(Rect(x + w, y + h, w, h), self.capacity)
        self.divided = True

    def insert(self, point):
        """Insert a point into the Quad Tree."""
        if not self.boundary.contains(point):
            return False

        if len(self.points) < self.capacity:
            self.points.append(point)
            return True
        else:
            if not self.divided:
                self.subdivide()

            if self.nw.insert(point) or self.ne.insert(point) or
self.sw.insert(point) or self.se.insert(point):
                return True
            return False

    def query(self, range, found_points):
        """Query the Quad Tree for points within a range."""
        if not self.boundary.intersects(range):

```

```

        return

    for point in self.points:
        if range.contains(point):
            found_points.append(point)

    if self.divided:
        self.nw.query(range, found_points)
        self.ne.query(range, found_points)
        self.sw.query(range, found_points)
        self.se.query(range, found_points)

# Example usage and performance measurement
def run_quad_tree_example():
    boundary = Rect(0, 0, 100, 100)
    qt = QuadTree(boundary, 4)

    for _ in range(1000):
        point = Point(random.uniform(-100, 100), random.uniform(-100, 100))
        qt.insert(point)

    query_range = Rect(0, 0, 50, 50) # Example query range
    found_points = []

    start_time = time.time()
    mem_usage_before = memory_usage(-1, interval=0.1, timeout=1)
    qt.query(query_range, found_points)
    mem_usage_after = memory_usage(-1, interval=0.1, timeout=1)
    end_time = time.time()

    print(f"Quad Tree Query: Execution time = {end_time - start_time}s, Memory
usage = {max(mem_usage_after) - mem_usage_before[0]}MB, Found points =
{len(found_points)}")

if __name__ == "__main__":
    run_quad_tree_example()

```

```

import pandas as pd
from rtree import index
from memory_profiler import memory_usage
import time
import random

```

```

def load_data(file_path):
    df = pd.read_csv(file_path)
    df['X'] = [random.uniform(0, 100) for _ in range(len(df))]
    df['Y'] = [random.uniform(0, 100) for _ in range(len(df))]
    return df

# Initialize R-tree with the dataset
def init_r_tree(data):
    p = index.Property()
    p.dimension = 2
    idx = index.Index(properties=p)

    # Insert data into the R-tree
    for i, row in data.iterrows():
        # The bounding box is a single point (x_min, y_min, x_max, y_max)
        idx.insert(i, (row['X'], row['Y'], row['X'], row['Y']),
obj=row.to_dict())

    return idx

# Sample query function for R-tree
def query_r_tree(tree, query_bounds):
    results = [n.object for n in tree.intersection(query_bounds, objects=True)]
    return results

# Performance Measurement Function
def measure_performance(func, *args, **kwargs):
    start_time = time.time()
    mem_usage_before = memory_usage(-1, interval=0.1, timeout=1)
    result = func(*args, **kwargs)
    mem_usage_after = memory_usage(-1, interval=0.1, timeout=1)
    end_time = time.time()
    execution_time = end_time - start_time
    max_memory = max(mem_usage_after) - mem_usage_before[0]
    return execution_time, max_memory, result

# Main function to run R-tree comparison
def run_r_tree_comparison(data_path):
    data = load_data(data_path)
    r_tree = init_r_tree(data)

    # Define a sample query bounds (adjust based on your dataset and needs)
    query_bounds = (0, 0, 50, 50) # Querying a rectangular area

```

```
    execution_time, memory_usage, result = measure_performance(query_r_tree,
r_tree, query_bounds)
    print(f"R-tree: Execution time = {execution_time}s, Memory usage =
{memory_usage}MB, Result count = {len(result)}")

if __name__ == "__main__":
    run_r_tree_comparison("scientists.csv")
```

```
import matplotlib.pyplot as plt

data_structures = ['KD-Tree', 'Simulated Range Tree', 'Quad Tree', 'R-tree']
result_counts = [268, 10, 256, 67]

plt.figure(figsize=(10, 6))
plt.bar(data_structures, result_counts, color='skyblue')
plt.title('Result Count Comparison for Different Spatial Data Structures')
plt.xlabel('Spatial Data Structure')
plt.ylabel('Result Count')
plt.xticks(rotation=45)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```